



Computer Vision and Image Processing

Lecture 14

Image Transformations

An **image transformation** is the process of changing the position, orientation, scale, or shape of an image. Unlike image enhancement (brightness, contrast, filtering), image transformation changes the **geometry** of the image.

Why are Image Transformations Needed?

- Panorama stitching
- Face alignment
- Image registration
- Satellite image matching
- Medical image registration
- Object tracking
- Augmented Reality
- OCR document correction

Types of Image Transformations

A. Intensity Transformation

- Brightness
- Contrast
- Gamma correction

Does NOT move pixels.

B. Geometric Transformation

- Translation
- Rotation
- Scaling
- Reflection
- Shearing

Translation

Translation means **moving an image from one location to another without changing its size or orientation.**

Examples:

- Moving an object to the right
- Shifting a camera frame
- Correcting image displacement
- The shape remains exactly the same.

Translation

Suppose a point is: $P(x, y)$

After translation: $x' = x + t_x$

$y' = y + t_y$

Where: t_x = horizontal movement

t_y = vertical movement

Assume a point: $P(3, 5)$

We move it : $t_x = 4$

$t_y = 2$

Therefore: $x' = 3 + 4 = 7$

$y' = 5 + 2 = 7$

New position: $P'(7, 7)$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Multiplying this out yields:

- $x' = (1 \cdot x) + (0 \cdot y) + (t_x \cdot 1) = x + t_x$
- $y' = (0 \cdot x) + (1 \cdot y) + (t_y \cdot 1) = y + t_y$
- $1 = (0 \cdot x) + (0 \cdot y) + (1 \cdot 1) = 1$

Scaling

Scaling is a linear geometric transformation that alters the size of an image by expanding or compressing its dimensions along the coordinate axes.

Two Types of Scaling:

1. Enlargement
2. Shrinking

Scaling

Suppose a point is: $P(x, y)$

After translation: $x' = S_x \cdot x$

$y' = S_y \cdot y$

Where:

S_x = scaling factor in x direction

S_y = scaling factor in y direction

Assume a point: $P(3, 5)$

We move it : $S_x = 4$

$S_y = 2$

Therefore: $x' = 3 \cdot 4 = 12$

$y' = 5 \cdot 2 = 10$

New position: $P'(12, 10)$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- **The Scale Factors:**

- If $s > 1$, the image **enlarges** (magnification/zoom-in).
- If $0 < s < 1$, the image **shrinks** (reduction/zoom-out).
- If $s = 1$, the image remains unchanged.
- If $s < 0$, it introduces a **reflection** (flipping) across the axis in addition to scaling.

- **Uniform vs. Non-Uniform Scaling:**

- If $s_x = s_y$, it is *uniform scaling*. The aspect ratio is preserved, meaning the object grows or shrinks symmetrically without distortion.
- If $s_x \neq s_y$, it is *non-uniform scaling*. The image will stretch or squash horizontally or vertically (e.g., turning a square into a rectangle), which distorts the original proportions.

Rotation

Rotation is a rigid-body geometric transformation that turns an image around a fixed center point by a specific angle θ . It preserves all lengths, areas, and internal angles, meaning the shape and size of the object remain completely unaltered.

Examples:

- Rotating a photo
- Correcting camera tilt
- Aligning objects

Rotation

- $x' = x\cos\theta - y\sin\theta$
- $y' = x\sin\theta + y\cos\theta$
- $R = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$

Rotate a Point by 90°

- Given: $P(2,1)$
- Angle: $\theta = 90^\circ$

- We know:

$$\cos 90 = 0$$

$$\sin 90 = 1$$

- Now:

- $x' = 2(0) - 1(1) \quad x' = -1$

- $y' = 2(1) + 1(0) \quad y' = 2$

- Therefore: $\boxed{P'(-1, 2)}$

Reflection

Reflection (often called flipping or mirroring) is a linear geometric transformation that mirrors an image across a specified axis (such as the X-axis, Y-axis, or origin). Like rigid transformations, it preserves lengths and internal angles, but it introduces a **chirality (handedness) reversal**—turning a "left-handed" orientation into a "right-handed" one (like looking in a mirror).

Reflection

A. Reflection Across the Y-Axis (Horizontal Flip)

If you flip an image horizontally, every pixel at position (x, y) moves to $(-x, y)$ relative to the axis of reflection:

$$x' = -x$$

$$y' = y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection

B. Reflection Across the X-Axis (Vertical Flip)

If you flip an image vertically, every pixel at position (x, y) moves to $(x, -y)$:

$$x' = x$$

$$y' = -y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection

C. Reflection Across the Origin

Flipping across both axes simultaneously ($x' = -x, y' = -y$) multiplies both scale factors by -1 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shearing

Shearing (or skewing) is a linear geometric transformation that pushes one part of an image in a certain direction while keeping the opposite part fixed, causing the shape to distort into a parallelogram. Unlike rotation or scaling, shearing alters the internal angles between lines.

Shearing is rarely used in isolation for creative effects, but it is heavily used as a **data augmentation** technique in deep learning (artificially slanting text or objects to train robust neural networks) and as an intermediate step in fast affine decomposition algorithms.

Shearing

A. Horizontal Shear (Shear along the X-axis)

In a horizontal shear, the X-coordinate of a pixel is shifted proportional to its Y-coordinate. The Y-coordinate remains entirely unaffected:

$$x' = x + sh_x \cdot y$$

$$y' = y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shearing

B. Vertical Shear (Shear along the Y-axis)

In a vertical shear, the Y-coordinate is shifted proportional to its X-coordinate, while the X-coordinate remains untouched:

$$x' = x$$

$$y' = sh_y \cdot x + y$$

Expressed in matrix form:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Image Warping in Computer Vision

Image warping is the process of transforming an image by changing the spatial locations of its pixels according to a mathematical mapping function.

In simple words : Image warping changes the shape, size, orientation, or perspective of an image by moving pixels from their original positions to new positions (by applying transformations).

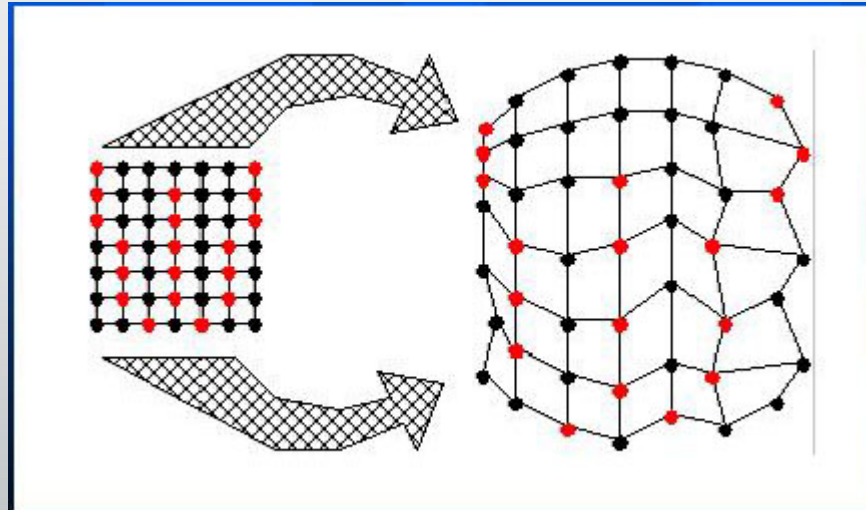


Image Warping in Computer Vision



1. Panorama Stitching



2. Face Alignment



3. Document Correction
(Deskewing)

Image Warping in Computer Vision

Forward Mapping: In forward mapping, every pixel from the original image is moved to a new position. The process: Source pixel $\rightarrow$ Destination pixel

Problem of Forward Mapping: Some pixels may not receive any value. Empty spaces are called : **holes** because the destination image contains pixels without intensity values.

Backward Mapping: Backward mapping works in the opposite direction. Instead of asking: Where should this source pixel go? it asks: From where should this destination pixel get its value? The process : Destination pixel $\rightarrow$ Find source pixel

Advantages:

- No holes
- Every output pixel gets a value
- Better image quality

Therefore, most computer vision libraries use backward warping.

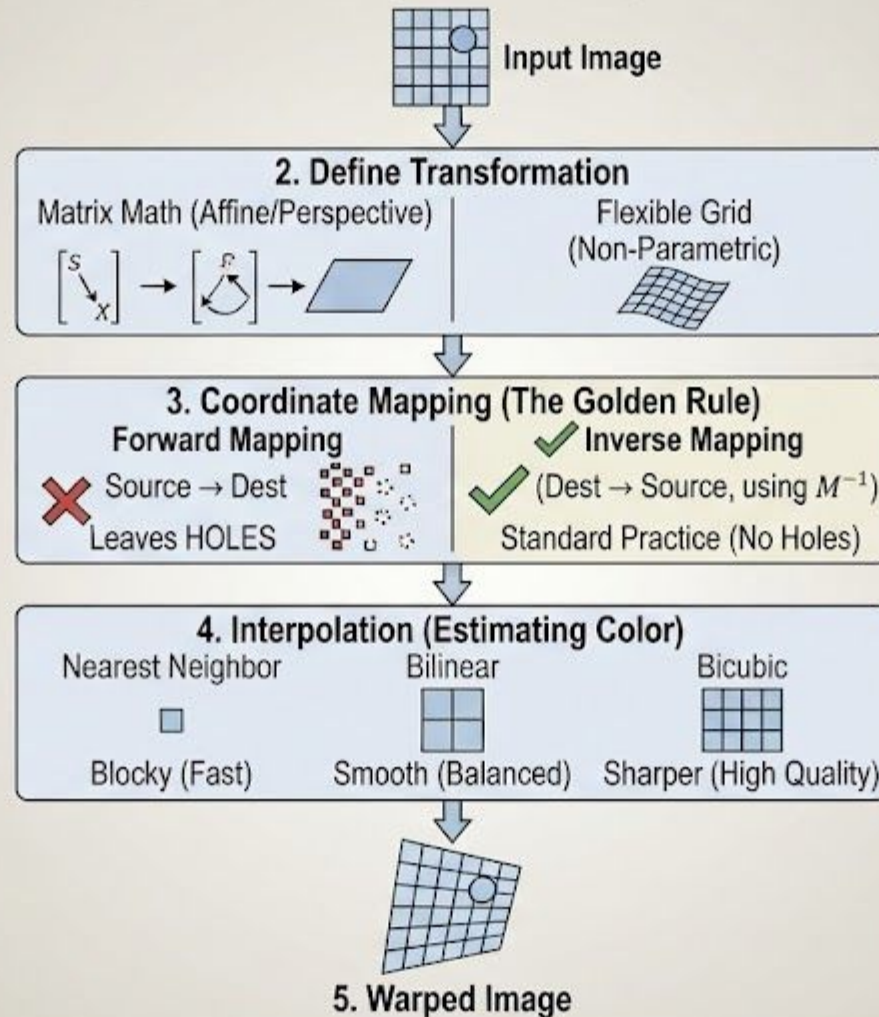
Image Warping in Computer Vision

Interpolation in Image Warping: After transformation, the calculated coordinate may not be an integer. Therefore, we need interpolation.

1. **Nearest Neighbour Interpolation**
2. **Bilinear Interpolation**
3. **Bicubic Interpolation**

Image Warping in Computer Vision

Simplified Image Warping Pipeline.



Affine vs Perspective

	Affine	Perspective
Think of it as	Moving/stretching an image	Changing camera viewpoint
Shape	Parallelogram	Trapezoid possible
Parallel lines	Stay parallel	May converge
Camera model	Approximate	Real camera projection
Common use	Alignment, rotation, scaling	Panorama, document correction



The End